Web Architecture

1. Introduction

Web architecture defines the structural design and operational behavior of web applications. It governs how different components such as clients, servers, and networks interact with each other to deliver web-based services. At its core, web architecture enables communication between a client (typically a web browser) and a server using the Hypertext Transfer Protocol (HTTP).

A strong understanding of web architecture is essential for developers, system administrators, and security professionals, as it directly impacts performance, scalability, reliability, and security of web applications

---

2. HTTP Status Codes

HTTP status codes are standardized response codes returned by a server to indicate the outcome of a client's request. They help clients understand whether a request was successful, failed, or requires further action.

2.1 Categories of HTTP Status Codes

HTTP status codes are grouped into five major categories:

1xx – Informational Responses (100–199)

These codes indicate that the request has been received and the process is continuing.

2xx – Successful Responses (200–299)

These codes confirm that the request was successfully received, understood, and processed by the server.

3xx – Redirection Messages (300–399)

These codes indicate that further action is required by the client, usually involving redirection to a different URL.

4xx – Client Error Responses (400–499)

These codes signal that the request contains errors or cannot be fulfilled due to issues on the client side.

5xx – Server Error Responses (500–599)

These codes indicate that the server failed to fulfill a valid request due to internal errors or temporary unavailability

---

2.2 Common HTTP Status Codes

200 OK

The request was successful and the server returned the expected response.

201 Created

The request was successful and resulted in the creation of a new resource on the server.

301 Moved Permanently

The requested resource has been permanently moved to a new URL, and future requests should use the new location.

302 Found

The resource is temporarily available at a different URL, but the original URL should still be used.

400 Bad Request

The server could not understand the request due to invalid syntax or malformed input.

401 Unauthorized

Authentication is required, and the client has not provided valid credentials.

403 Forbidden

The client is authenticated but does not have permission to access the requested resource.

404 Not Found

The server cannot find the requested resource, usually due to an incorrect or non-existent URL.

500 Internal Server Error

The server encountered an unexpected condition that prevented it from fulfilling the request.

503 Service Unavailable

The server is currently unable to handle the request, often due to maintenance or overload

---

3. HTTP Methods

HTTP methods define the type of action a client wants to perform on a resource. They are a fundamental part of RESTful web services and APIs.

GET

Retrieves data from the server without making any changes to the resource.

POST

Sends data to the server to create a new resource.

PUT

Replaces or updates an existing resource with new data.

DELETE

Removes a resource from the server.

PATCH

Applies partial modifications to a resource rather than replacing it entirely.

HEAD

Retrieves only the response headers without the response body.

OPTIONS

Returns the HTTP methods supported by a server for a specific resource.

TRACE

Echoes the received request back to the client, primarily used for debugging.

CONNECT

Establishes a tunnel to the server, commonly used for secure HTTPS connections

---

4. Common HTTP Request Headers

HTTP request headers provide additional context and metadata about the client request.

Host

Specifies the domain name of the server being accessed.

User-Agent

Identifies the client software making the request, such as a browser or API client.

Accept

Indicates the media types that the client can process, such as JSON or XML.

Authorization

Carries credentials used for authentication.

Content-Type

Defines the format of the request body.

Cookie

Stores session data and user-related information.

Accept-Language

Specifies the preferred language for the response.

Accept-Encoding

Indicates supported compression methods like gzip or deflate.

Connection

Controls whether the network connection remains open after the request.

Cache-Control

Specifies caching directives.

Content-Length

Indicates the size of the request body.

Origin

Identifies the origin of the request, often used in Cross-Origin Resource Sharing (CORS)

---

5. Common HTTP Response Headers

HTTP response headers provide metadata about the server’s response.

Content-Type

Specifies the format of the response body.

Content-Length

Indicates the size of the response body.

Set-Cookie

Sends cookies from the server to the client.

Cache-Control

Defines how responses should be cached.

ETag

Used for versioning and cache validation.

Location

Specifies a new URL during redirection.

Server

Identifies the web server software handling the request.

Date

Indicates when the response was generated.

Last-Modified

Shows when the resource was last updated.

Content-Encoding

Specifies compression applied to the response.

Expires

Defines when the response content becomes invalid.

Strict-Transport-Security (HSTS)

Forces browsers to use HTTPS for all future requests to the domain

---

6. Conclusion

Web architecture relies heavily on standardized HTTP methods, status codes, and headers to ensure reliable communication between clients and servers. A thorough understanding of these components improves application development, debugging efficiency, performance optimization, and security posture.

By mastering these core web architecture concepts, developers and security professionals can build more robust, scalable, and secure web applications